Lecture 4 — Tree Decompositions: Heavy-Light, Centroid, Small-to-Large

> National Championship Prep · Prerequisites: trees and tree DP (week 20), LCA and Euler tour (week 22), segment trees and lazy propagation (weeks 23, 25) · Estimated time: 2 × 90 min

1. Overview

Week 22 showed how an Euler tour flattens a tree so that **subtrees** become contiguous ranges. Championship problems ask for more:

Query shape	Example	Tool
aggregate / update on a **path** u–v, with updates	"max on path", "add x to every node on the path"	Heavy-light decomposition (HLD)
something about **all paths** or **distances** in the tree	"count paths of length k", "nearest marked vertex"	Centroid decomposition
per-subtree statistics that are expensive to combine	"number of distinct colors in each subtree"	Small-to-large merging

All three rest on the same idea: **a logarithmic bound coming from halving sizes.**

- HLD: walking up from any vertex you cross at most $\log_2 n$ light edges, because each light edge at least doubles the subtree size.
- Centroid decomposition: removing a centroid leaves components of size $\le n/2$, so the recursion depth is $\le \log_2 n$.
- Small-to-large: an element moves only into a set at least as large as its own, so its set size at least doubles with every move — at most $\log_2 n$ moves.

2. Heavy-light decomposition

2.1 Definitions

Root the tree. For each non-leaf vertex v, its **heavy child** is the child with the largest subtree (ties arbitrary). The edge to the heavy child is **heavy**; all other child edges are **light**. Heavy edges form vertex-disjoint **chains** (heavy paths); every vertex belongs to exactly one chain, identified by its topmost vertex `head[v]`.

Lemma. Every root-to-vertex path contains at most $\lfloor \log_2 n \rfloor$ light edges.

Proof. If (p, c) is light then p has a heavy child h ≠ c with size(h) ≥ size(c), so size(p) ≥ 1 + size(c) + size(h) > 2·size(c). Going up a light edge more than doubles the subtree size, and sizes never exceed n. ∎

Consequently any path u–v intersects **O(log n) chains**: it goes up from u to the LCA and from v to the LCA, and each upward walk switches chains only at light edges.

2.2 Positions

Do a DFS that always visits the **heavy child first**, and number vertices in preorder (`pos[v]`). Then:

- every chain occupies a contiguous range of positions (the heavy child gets `pos[v] + 1`);
- every subtree occupies `[pos[v], pos[v] + size(v) − 1]`, exactly as in week 22.

Build one segment tree over positions. Path queries decompose into O(log n) chain segments, each a contiguous range: **O(log² n)** per path query/update with a segment tree (O(log n) per range). Subtree queries cost O(log n). Lazy propagation (week 25) works unchanged for range updates on paths.

2.3 Path query

```
while head[a] != head[b]:
    if depth[head[a]] < depth[head[b]]: swap(a, b)
    process range [pos[head[a]], pos[a]]      # the deeper chain's part
    a = parent[head[a]]
# now on the same chain
if depth[a] > depth[b]: swap(a, b)
process range [pos[a], pos[b]]                 # a is the LCA
```

Always lift the chain whose **head** is deeper — not the deeper vertex. Lifting by vertex depth can jump past the LCA.

The loop also computes the LCA as a by-product (the final `a`), so HLD doubles as an O(log n) LCA structure with O(n) memory.

2.4 Values on edges

If weights are on edges, store the weight of edge (parent(v), v) at vertex v. A path query then must **exclude the LCA**: the last range becomes `[pos[a] + 1, pos[b]]` (skip it when `a == b`). SPOJ QTREE is the classic example.

2.5 Non-commutative aggregates

If the combination is not commutative (e.g. "composition of linear functions along the path", "maximum subarray sum along the path"), the order matters: the u-side segments are traversed upward (reverse of position order) and the v-side segments downward. Keep two accumulators, one per side, and combine them at the end with the u-side reversed.

3. Centroid decomposition

3.1 Centroids

A **centroid** of a tree with n vertices is a vertex whose removal leaves components of size ≤ n/2.

Existence and how to find one. Start at any vertex v. If some neighbor's component (relative to v) has size > n/2, move to that neighbor. The component sizes in the direction we came from are < n/2, so we never move back; the walk ends at a centroid. With subtree sizes from one DFS, this is O(n).

3.2 The decomposition

```
decompose(component):
    c = centroid(component)
    process all paths through c          # problem-specific, O(size) or O(size log size)
    mark c removed
    for each remaining component: decompose(it)
```

Component sizes at least halve at each level, so the **centroid tree** (each centroid's parent is the centroid of the enclosing component) has depth ≤ $\log_2 n + 1$, and each vertex belongs to O(log n) components. If processing a component is linear in its size, the total is **O(n log n)**.

Key property. For any two vertices u, v, consider the first centroid c removed among the vertices of the path u–v. Then c lies on the path, u and v were in the same component when c was chosen, and c is a common ancestor of u and v in the centroid tree. So **every path is "handled" at exactly one centroid**, and dist(u, v) = dist(u, c) + dist(c, v).

3.3 Counting paths

Count paths with exactly k edges (CSES 2080 — Fixed-Length Paths I). At centroid c, process its child subtrees one at a time. Keep `cnt[d]` = number of already-processed vertices at depth d from c (with `cnt[0] = 1` for c itself). For each vertex at depth d in the current subtree, add `cnt[k − d]`; only after finishing the subtree, add its depths to `cnt`. Processing subtree by

subtree guarantees the two endpoints are in different subtrees, so the path really passes through c. Reset `cnt` using only the depths you touched — never with a full `memset` — to keep the work proportional to the component size.

3.4 Online queries: nearest marked vertex

Codeforces 342E — Xenia and Tree: vertex 1 is red; queries paint a vertex red or ask the distance to the nearest red vertex.

For every centroid c keep `best[c]` = min distance from c to a red vertex in c's component. For every vertex v precompute the list of its centroid ancestors with `dist(v, c)` (O(n log n) memory total).

- Paint v: for each centroid ancestor c, `best[c] = min(best[c], dist(v, c))`.
- Query v: answer = min over centroid ancestors c of `best[c] + dist(v, c)`.

Correctness. Every term is the length of some walk from v to a red vertex, so it is ≥ the true answer. Conversely, let r be the nearest red vertex and c the first-removed centroid on the path v–r: c is a common centroid ancestor, `best[c] ≤ dist(c, r)`, and `dist(v, c) + dist(c, r) = dist(v, r)`. Both operations cost O(log n).

This "store something at each centroid ancestor" pattern solves many online distance problems.

3.5 Centroid decomposition as a structure

Codeforces 321C — Ciel the Commander asks to assign ranks A–Z so that any two vertices with the same rank have a higher rank on the path between them. The centroid tree's depth is ≤ $\log_2(10^5) + 1 < 26$: give each vertex the letter of its depth in the centroid tree.

4. Small-to-large merging

4.1 The technique

Many subtree problems want, for every vertex v, a statistic of the multiset of values in its subtree: number of distinct colors, most frequent color (Codeforces 600E — Lomsat gelral), count of vertices at each depth (Codeforces 1009F — Dominant Indices).

Computing each subtree's set from scratch is $O(n^2)$. Instead, build each vertex's set by **merging its children's sets into the largest one**: keep the largest child's container (by swapping — O(1) for STL containers), and insert the elements of the others into it.

Lemma. Each element is inserted O(log n) times in total.

Proof. When an element moves, it moves from a set of size s into a set of size ≥ s, so the size of the set containing it at least doubles. Sizes are ≤ n. ∎

With `std::set`/`std::map` that is **O(n log² n)**; with `unordered_map` or arrays, O(n log n) expected.

4.2 Answering at the right time

The answer for v must be read **after** all children are merged into v's container and **before** v's container is merged into its parent (after which it is destroyed or reused). In a post-order loop this is natural. If queries ask about v with a parameter (e.g. "how many vertices of color c in subtree v"), group queries by vertex and answer them at that moment.

4.3 Relatives

- **DSU on tree (sack)**: the same bound, with a single global counting array and the heavy child's contribution kept — covered in worldfinals-04.
- **Merging other structures**: Li Chao trees (CF 932F, Lecture 3), segment trees (segment tree merging), treaps (Lecture 2). The same doubling argument applies whenever merge cost is proportional to the smaller structure's size (times a log).

5. Choosing a tool

Problem	Tool
subtree query/update only	Euler tour + segment tree (week 22)
path query/update, online, with updates	HLD (+ lazy segtree)
path sum, point updates only	Euler tour + Fenwick with root-path sums (CSES 1138), simpler than HLD
count/optimize over all paths	centroid decomposition
online distance to a dynamic set	centroid decomposition (ancestor lists)
per-subtree distinct/mode/depth statistics	small-to-large (or DSU on tree)

6. Reference implementations

All programs compile with `g++ -std=c++17 -O2 -Wall`, were stress-tested against brute force on random small trees (paths, caterpillars and random shapes), and run in well under a second on $n = 2 \cdot 10^5$ path-shaped trees.

6.1 HLD: path maximum with point updates (CSES 2134), plus subtree maximum

Input: `n q`, the values, n − 1 edges, then queries `1 v x` (set), `2 a b` (path max), `3 v` (subtree max; not part of the CSES task).

```
#include <bits/stdc++.h>
using namespace std;

// CSES 2134 "Path Queries II": point assignment, maximum on a path.
// Heavy-light decomposition + iterative segment tree: O(log^2 n) per path query.
// Extra query type 3 (not in CSES): maximum over the subtree of v (root = 1), O(log n).
int n;
vector<vector<int>> adj;
vector<int> par, dep, heavy, head, pos, sz;
vector<int> seg;  // iterative max segment tree over positions 0..n-1

void segUpdate(int p, int v) {
    for (seg[p += n] = v; p > 1; p >>= 1) seg[p >> 1] = max(seg[p], seg[p ^ 1]);
}
int segQuery(int l, int r) {  // inclusive [l, r]
    int res = INT_MIN;
    for (l += n, r += n + 1; l < r; l >>= 1, r >>= 1) {
        if (l & 1) res = max(res, seg[l++]);
        if (r & 1) res = max(res, seg[--r]);
    }
    return res;
}

void build(int root) {
    par.assign(n + 1, 0); dep.assign(n + 1, 0); heavy.assign(n + 1, -1);
    head.assign(n + 1, 0); pos.assign(n + 1, 0); sz.assign(n + 1, 1);
    // iterative DFS order (avoids stack overflow on path-like trees)
    vector<int> order; order.reserve(n);
    vector<int> st = {root}; par[root] = 0;
    while (!st.empty()) {
        int v = st.back(); st.pop_back(); order.push_back(v);
        for (int u : adj[v]) if (u != par[v]) { par[u] = v; dep[u] = dep[v] + 1;
st.push_back(u); }
    }
    for (int i = n - 1; i >= 0; i--) {          // children before parents
        int v = order[i];
        if (par[v]) {
            sz[par[v]] += sz[v];
            if (heavy[par[v]] == -1 || sz[v] > sz[heavy[par[v]]]) heavy[par[v]] = v;
        }
    }
    // heavy-first preorder: every heavy chain AND every subtree is a contiguous range
    int cur = 0;
    st = {root}; head[root] = root;
    while (!st.empty()) {
        int v = st.back(); st.pop_back();
        pos[v] = cur++;
```

```
        for (int u : adj[v]) if (u != par[v] && u != heavy[v]) { head[u] = u;
st.push_back(u); }
        if (heavy[v] != -1) { head[heavy[v]] = head[v]; st.push_back(heavy[v]); }  //
popped next
    }
}

int pathMax(int a, int b) {
    int res = INT_MIN;
    while (head[a] != head[b]) {
        if (dep[head[a]] < dep[head[b]]) swap(a, b);   // lift the chain whose head is
deeper
        res = max(res, segQuery(pos[head[a]], pos[a]));
        a = par[head[a]];
    }
    if (dep[a] > dep[b]) swap(a, b);
    return max(res, segQuery(pos[a], pos[b]));        // same chain now
}

int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int q; cin >> n >> q;
    vector<int> val(n + 1);
    for (int i = 1; i <= n; i++) cin >> val[i];
    adj.assign(n + 1, {});
    for (int i = 0; i < n - 1; i++) { int a, b; cin >> a >> b; adj[a].push_back(b);
adj[b].push_back(a); }
    build(1);
    seg.assign(2 * n, INT_MIN);
    for (int v = 1; v <= n; v++) segUpdate(pos[v], val[v]);
    while (q--) {
        int t, a; cin >> t >> a;
        if (t == 1) { int x; cin >> x; segUpdate(pos[a], x); }
        else if (t == 2) { int b; cin >> b; cout << pathMax(a, b) << " "; }
        else cout << segQuery(pos[a], pos[a] + sz[a] - 1) << " ";
    }
    cout << "\n";
}
```

Both DFS passes are iterative: a path-shaped tree with $n = 2 \cdot 10^5$ would otherwise risk a stack overflow on judges with small stacks. The second pass pushes the heavy child last so that it is popped (and numbered) immediately after its parent.

6.2 Centroid decomposition: Xenia and Tree (CF 342E)

```
#include <bits/stdc++.h>
using namespace std;

// Codeforces 342E "Xenia and Tree": node 1 starts red.
// "1 v": paint v red.  "2 v": distance from v to the nearest red node.
// Centroid decomposition: every node has O(log n) centroid ancestors; we store
// the distance to each of them, so both operations cost O(log n).
```

```cpp
int n;
vector<vector<int>> adj;
vector<bool> removed;
vector<int> subSz;
vector<vector<pair<int,int>>> anc;   // anc[v] = list of (centroid c, dist(v, c)),
outermost first
vector<int> best;                    // best[c] = min distance from c to a red node in
c's component

int calcSize(int v, int p) {
    subSz[v] = 1;
    for (int u : adj[v]) if (u != p && !removed[u]) subSz[v] += calcSize(u, v);
    return subSz[v];
}
int findCentroid(int v, int p, int total) {
    for (int u : adj[v])
        if (u != p && !removed[u] && subSz[u] * 2 > total) return findCentroid(u, v,
total);
    return v;
}
void collect(int v, int p, int d, int c) {   // record dist(v, c) for every v in c's
component
    anc[v].push_back({c, d});
    for (int u : adj[v]) if (u != p && !removed[u]) collect(u, v, d + 1, c);
}
void decompose(int v) {
    int c = findCentroid(v, 0, calcSize(v, 0));
    collect(c, 0, 0, c);
    removed[c] = true;
    for (int u : adj[c]) if (!removed[u]) decompose(u);
}

void paint(int v) { for (auto [c, d] : anc[v]) best[c] = min(best[c], d); }
int query(int v) {
    int res = INT_MAX;
    for (auto [c, d] : anc[v]) if (best[c] != INT_MAX) res = min(res, best[c] + d);
    return res;
}

int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int m; cin >> n >> m;
    adj.assign(n + 1, {});
    for (int i = 0; i < n - 1; i++) { int a, b; cin >> a >> b; adj[a].push_back(b);
adj[b].push_back(a); }
    removed.assign(n + 1, false); subSz.assign(n + 1, 0); anc.assign(n + 1, {});
    best.assign(n + 1, INT_MAX);
    decompose(1);
    paint(1);
    while (m--) {
        int t, v; cin >> t >> v;
        if (t == 1) paint(v); else cout << query(v) << "\n";
    }
}
```

`calcSize`, `findCentroid` and `collect` are recursive; recursion depth is bounded by the component's height. That is fine on Codeforces (large stack), but for very deep trees on other judges, convert them to iterative BFS order the same way as in the HLD code.

6.3 Small-to-large: Distinct Colors (CSES 1139)

```cpp
#include <bits/stdc++.h>
using namespace std;

// CSES 1139 "Distinct Colors": number of distinct colors in every subtree.
// Small-to-large: each node keeps a set; merge the smaller set into the larger.
// An element moves only into a set at least twice the size of its old one,
// so it moves O(log n) times: O(n log^2 n) total with std::set.
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n; cin >> n;
    vector<int> color(n + 1);
    for (int i = 1; i <= n; i++) cin >> color[i];
    vector<vector<int>> adj(n + 1);
    for (int i = 0; i < n - 1; i++) { int a, b; cin >> a >> b; adj[a].push_back(b);
adj[b].push_back(a); }

    vector<int> par(n + 1, 0), order;            // iterative DFS order
    order.reserve(n);
    vector<int> st = {1};
    while (!st.empty()) {
        int v = st.back(); st.pop_back(); order.push_back(v);
        for (int u : adj[v]) if (u != par[v]) { par[u] = v; st.push_back(u); }
    }
    vector<set<int>> s(n + 1);
    vector<int> ans(n + 1);
    for (int i = n - 1; i >= 0; i--) {           // children are processed before parents
        int v = order[i];
        s[v].insert(color[v]);
        ans[v] = s[v].size();                    // all children already merged into s[v]
        int p = par[v];
        if (p) {
            if (s[p].size() < s[v].size()) swap(s[p], s[v]);   // O(1) swap of contents
            for (int c : s[v]) s[p].insert(c);
            set<int>().swap(s[v]);                             // free memory
        }
    }
    for (int i = 1; i <= n; i++) cout << ans[i] << " \n"[i == n];
}
```

`swap` of two `std::set`s is O(1): it exchanges internal pointers. Never copy the larger set.

7. Common mistakes

- **HLD: lifting the deeper vertex instead of the vertex with the deeper chain head.** Produces ranges that cross the LCA.
- **HLD positions not heavy-first.** Chains are then not contiguous, and the range `[pos[head[a]], pos[a]]` contains unrelated vertices. If subtree queries are also needed, the numbering must be a heavy-first *preorder*.
- **Edge weights: including the LCA's value.** Use `pos[a] + 1` in the final segment.
- **Non-commutative aggregates combined in the wrong order** along the u-side of the path.
- **Centroid: recomputing sizes from a stale root.** Sizes must be recomputed inside each component (only over non-removed vertices) before finding its centroid.
- **Centroid path counting: pairs within the same child subtree.** Add a subtree's depths to the counter only after querying all of its vertices.
- **Resetting a global array with `memset` per centroid** — O(n) per centroid gives $O(n^2)$. Clear only what you touched.
- **Small-to-large: merging the larger set into the smaller.** Always compare sizes and swap first.
- **Reading the answer for v after v's set was merged into its parent.**
- **Recursion depth** on path-shaped trees ($n = 2 \cdot 10^5$): use iterative DFS or increase the stack.

8. Practice problems

Heavy-light decomposition

1. CSES 2134 — Path Queries II
2. CSES 1138 — Path Queries (also solvable with an Euler tour; do both)
3. SPOJ QTREE — Query on a tree (edge weights)
4. Codeforces 343D — Water Tree (path and subtree assignments)

Centroid decomposition 5. Codeforces 342E — Xenia and Tree 6. Codeforces 321C — Ciel the Commander 7. CSES 2080 — Fixed-Length Paths I 8. CSES 2081 — Fixed-Length Paths II (lengths in $[k_1, k_2]$; prefix sums or a Fenwick tree over depths) 9. IOI 2011 — Race (weighted path of length exactly K with fewest edges) 10. Codeforces 715C — Digit Tree (centroid + modular arithmetic)

Small-to-large 11. CSES 1139 — Distinct Colors 12. Codeforces 600E — Lomsat gelral 13. Codeforces 1009F — Dominant Indices 14. Codeforces 932F — Escape Through Leaf (small-to-large on Li Chao trees, from Lecture 3)

9. Further reading

- cp-algorithms: Heavy-light decomposition.
- USACO Guide: Heavy-Light Decomposition, Centroid Decomposition, Small-to-Large Merging.
- D. D. Sleator and R. E. Tarjan, "A data structure for dynamic trees" (1983) — the origin of heavy/light edges, later developed into link-cut trees (worldfinals-04).
- Next: Lecture 5 (*Flows II*). The World Finals track continues trees with link-cut trees, virtual trees and DSU on tree (worldfinals-04).